Pegasus: Probabilistic Performance Guarantees for Agentic Provisioning of Software Services

Boris Sedlak, *Member, IEEE*, Milos Gravara, *Member, IEEE*, Víctor Casamayor Pujol, *Member, IEEE*, Stefan Nastic, *Member, IEEE*, Schahram Dustdar, *Fellow, IEEE*

***Abstract*—**
TODO: Check hierarchical setup with synthetic data.

***Index Terms*—Performance Guarantees, Service Quality, Gaussian Processes, Performance Modeling, Service Composition**

I. Introduction

Large Language Models (LLMs) have risen as universal interpretation and generation engines for text-based contents, introducing a major disruption for software development [6]. Traditionally, computing services were developed manually according to stakeholder requirements, and operated in close-loop controls to continuously meet operational thresholds. Within that context, LLMs appeared as a fast and cost-efficient enabler for autonomous software development and deployment, opening a new branch of service research [2]: automatic generation [5], [13] and recomposition [20] of software services. The combined vision from these works is clear: a *future internet*, where agents aim to continuously satisfy high-level user intents by generating, composing, and reconfiguring software applications and their services [1].

Traditional service-oriented architectures already provide clear roles and interfaces to operate such systems [21]: the service provider and the service consumer establish Service Level Agreements (SLAs) that specify the expected quality through measurable Service Level Objectives (SLOs). In this future internet, applications will be tailored to individual clients and their SLOs [23]. However, creating this vast space of possible application also means that potential compositions or configurations will often lack end-to-end benchmarks *prior to deployment*. It is neither possible nor cost-efficient to benchmark this search space exhaustively before offering a solution to a consumer; hence, service providers require probabilistic models for providing guarantees about the quality of different service compositions and configurations before deployment.

Applications themselves can be recomposed into systems of systems [19]; to decrease the overall complexity, individual systems (or services) must provide clear interfaces and guarantees. So suppose we need to ensure that one system's latency is below a threshold, it is not enough to *estimate* that the tail latency (e.g., the P_{99} percentile) will be below the threshold, but the application must *guarantee* this with a certain likelihood. To avoid SLA violations, providers could simply give increasingly conservative performance estimates, which could only be fulfilled by provisioning more resources, increasing the price for the consumer, and wasting the provider's resources. The challenge is thus to make the performance guarantee as *tight* as possible. For a service composition S, also called an application, with topology $\{s_1 \rightarrow s_2 \rightarrow \dots\}$, a provider needs to be able to accurately answer the following questions:

- What performance could we guaranteed for this application, before conducting any end-to-end benchmark?
- What performance could we guarantee if we replaced a service (e.g., s_1) with another one (e.g., s_x) that was not benchmarked end-to-end in this exact application?
- What performance could we guarantee if we reconfigured an individual service (e.g., s_1) to a configuration that was not benchmarked end-to-end in this application?

While composition and configuration are operationally decoupled—services can be reconfigured without altering the topology, and vice versa—they are logically entangled: estimating composite performance requires understanding individual service dynamics under limited empirical observations. Optimization techniques like branch-and-bound can reduce the candidate space, but evaluating candidate solutions remains the core bottleneck. Learning-based optimizers must contend with varying degrees of empirical evidence, ranging from well-observed service instances to completely unseen configurations and compositions.

TODO: Add summary paragraph introducing the contribution/solution: State the gap in existing work (e.g., deterministic code contracts vs. black-box microservices) and summarize the proposed PPG framework combining SVGPs and Stochastic Network Calculus.

II. Preliminaries

A. Motivating Example

Consider a customer that requests a real-time drone monitoring application, which can be structured as a four-stage pipeline: (1) video stream ingestion from the drone, (2) object detection along the trajectory, (3) trajectory adjustment calculation, and (4) telemetry reporting to an external traffic planner. Logically, this application is composed of distinct, chained microservices $S = \{s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4\}$.

Each service in this pipeline can be configured along multiple dimensions, such as allocated hardware resources

B. Sedlak, P. Raith, A. Morichetta, and S. Dustdar are with the Distributed Systems Group, TU Wien, Vienna, Austria (e-mail: b.sedlak@dsg.tuwien.ac.at; p.raith@dsg.tuwien.ac.at; a.morichetta@dsg.tuwien.ac.at; dustdar@dsg.tuwien.ac.at).

V. Casamayor Pujol is with the Department of Engineering, Universitat Pompeu Fabra, Barcelona, Spain and S. Dustdar is with ICREA (e-mail: v.casamayor@upf.edu; schahram.dustdar@upf.edu).

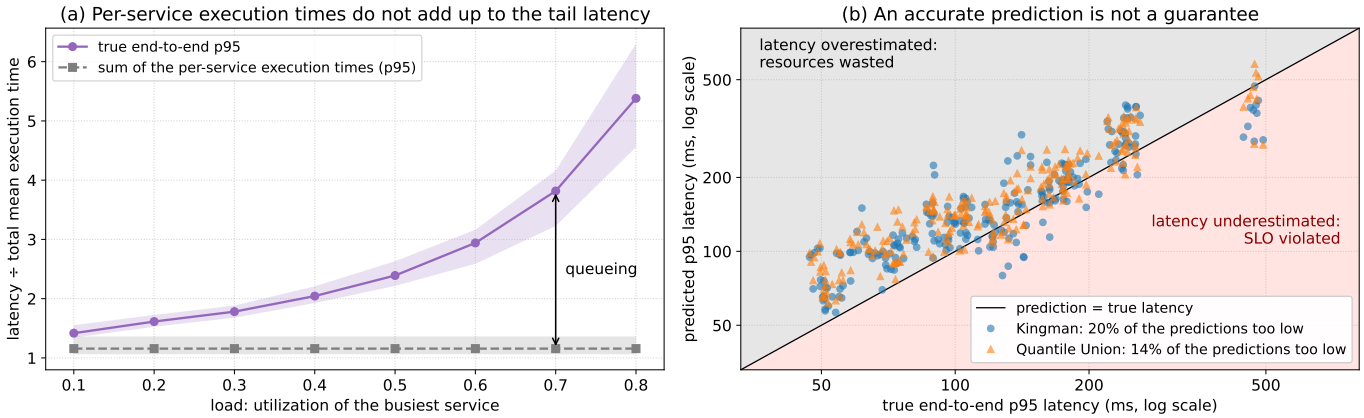


Fig. 1: Two reasons why per-service performance models do not yield an end-to-end guarantee, shown for a pipeline of three real services (QR-code detector → object detector → face detector) under Poisson arrivals; latencies are obtained by replaying the measured processing times of each service through FIFO queues. **(a)** The true P_{95} of the end-to-end latency and the sum of the predicted per-service execution times, both relative to the total mean execution time of the pipeline, for increasing load (median and quartiles over 48 configurations). The gap between the two is the queueing delay; it grows from a factor of 1.2 at a utilization of 0.1 to a factor of 4.7 at 0.8, so no constant correction closes it. **(b)** Predicted versus true P_{95} for unseen configurations (5 training sets, 240 predictions per method) for two predictors that account for queueing: a queueing formula on learned execution times (*Kingman*) and learned per-service latency quantiles (*Quantile Union*). Although both overestimate the latency for the median configuration, they underestimate it for 20% and 14% of the configurations, respectively, and their output does not reveal for which ones.

(e.g., CPU cores r) and internal service parameters (e.g., object detection model resolution or batch size c). While these configuration knobs dictate execution time, the customer is solely interested in end-to-end *performance* (e.g., total latency ≤ 100 ms) and overall *cost* (e.g., minimize resources).

Furthermore, external systems like the traffic planner pose strict workload bounds (e.g., updates must arrive consistently every 100 ms). Adhering to these bounds in distributed systems typically relies on reporting high-percentile tail latencies (e.g., P_{90} or P_{99}). However, this specific combination of pipeline topology and configuration has never been executed and benchmarked end-to-end prior to deployment; the provider only has measurements of the individual services, collected in isolation or in other applications. The following section shows why these measurements alone do not yield the required guarantee.

B. Research Challenges

Figure 1 shows, for a pipeline of three real services, why measurements of the individual services are not sufficient with existing techniques, and motivates the two challenges that this paper addresses.

Challenge #1: Composing Tail Latencies under Load. The end-to-end latency of a composition is not the sum of what its services need in isolation. Tail percentiles do not add up (i.e., $P_{99}(s_1 + s_2) \neq P_{99}(s_1) + P_{99}(s_2)$), and, more importantly, items wait in the queues between the services. This waiting time depends on the workload and on the combination of services, so it cannot be observed at an individual service: in Figure 1a, the sum of the per-service execution times explains most of the end-to-end P_{95} at low load, but less than a quarter of it at high load. Measuring the composition end to end would capture the queueing, but every candidate composition and configuration would have to be deployed and benchmarked first, which is exactly what must be avoided. The challenge is therefore to derive the end-to-end tail latency of a composition, including its queueing, purely from observations of the individual services and a description of the workload.

Challenge #2: Guaranteeing What Was Never Observed. An SLO requires a bound, i.e., a latency that is exceeded with at most a given probability, whereas performance models are trained to be accurate, i.e., close to the measured value on average. For an unseen configuration, such a prediction can err in either direction: an overestimated latency wastes resources, an underestimated one violates the SLO. Figure 1b shows that even predictors that overestimate the latency for most configurations still underestimate it for roughly one in six, and their output does not distinguish a configuration that is well covered by observations from one that is far away from all of them. The challenge is therefore to turn limited evidence about a service into a bound that separates the inherent variability between items (aleatoric uncertainty) from the ignorance of the model (epistemic uncertainty), and that becomes more conservative exactly where evidence is missing.

C. Background and Related Work

We group related work by how it obtains the performance of an application: (1) analytical models derive it from a formal description of the system, (2) learned models fit it to measurements of the entire application, and (3) compositional models assemble it from models of the individual services. For each group, we explain *how* it models performance and *why* it leaves the presented research challenges open.

1) Analytical Performance Models: Queueing theory is a classical tool for relating workload and capacity to response time [14]. For instance, Gandhi et al. [11] model the performance of individual servers as M/G/1 queues[1], and use this model for finding a power allocation that minimizes overall response time. Layered Queueing Networks [9] extend queueing networks to software services that hold one resource while waiting for another, as in nested client–server calls. Both target mean response times in steady state rather than tail probabilities (Challenge #2). Network calculus bounds delays instead of averaging them: Duan [4] describes network and compute services through latency–rate capability profiles, merges these profiles into one end-to-end profile, and derives the maximum delay of the composite service. This bound is deterministic, i.e., it covers the worst case and thus becomes overly conservative for services with variable execution times; Stochastic Network Calculus (SNC) [8], on which we build, instead bounds the *probability* of exceeding a given delay. SNC assumes that the MGFs of arrivals and service exist; an extension to heavy-tailed traffic replaces them by power-law envelopes at the price of considerably looser bounds [18].

Closest in spirit, Iyer et al. [15] compute *performance contracts* for software network functions without executing them: symbolic analysis of the source code yields conservative upper bounds on the memory accesses and execution cycles per packet, expressed as functions of workload-dependent variables; contracts for chains of network functions are computed recursively. As with worst-case execution time analysis [26], this requires access to the source code, whereas microservices are typically deployed as opaque containers. Moreover, these contracts bound the processing effort per packet, not waiting time that accumulates under load (Challenge #1).

Common to all analytical models is that their parameters (e.g., service rates, branching probabilities, or code paths) must be supplied for the exact system under study. None of them expresses how reliable a prediction is for a configuration that was never measured (Challenge #2).

2) Learning Models of Entire Applications: A second line of work treats the application as one big black box and learns its end-to-end performance from empirical measurements; we call this the *top-down* approach. *Ernest* [25] fits a small parametric model to short profiling runs of an analytics job on samples of the input data, and extrapolates the running time to larger datasets and clusters. *CherryPick* [3] replaces the fixed model structure with a Gaussian Process (GP) and uses its confidence intervals to choose the next cloud configuration to benchmark; the search stops as soon as the model can identify the best configuration. *Microscaler* [28] similarly combines Bayesian optimization (BO) with a heuristic to find the number of instances at which a microservice meets its SLA. Production systems are more conservative: Google's *Autopilot* [22] sets resource limits from moving-window statistics of past usage plus a safety margin, optionally choosing among many such window models by their historical cost.

[1]Note that M implies a *Markovian* (or Poisson) arrival process, and G a *General* service process, i.e., one that can be described through a mean and variance. *Pegasus* supports a wider range of arrival processes, but requires a sub-Gaussian service process (e.g., Gaussian or bounded execution times).

Top-down models are simple in the sense that they require no knowledge of the application's internal structure: a single model maps the configuration to the end-to-end performance. This is also their limitation: replacing a service or changing the topology changes the modeled function itself, so the end-to-end performance must be benchmarked again (Challenge #1). Furthermore, where uncertainty is modeled at all, as in *CherryPick*, it steers the search towards a good configuration but is not converted into a latency guarantee (Challenge #2).

3) Compositional Models: The *bottom-up* alternative predicts the quality of a composition from its constituent services, which has a long tradition in service-oriented computing. Gallotti et al. [10] translate activity diagrams into stochastic models for a probabilistic model checker; the execution time and reliability of each service are annotations supplied at design time. Wu et al. [27] use a hidden Markov model to predict whether a composition of Cloud services will satisfy the requested QoS, and Fanjiang et al. [7] evolve time-series predictors that forecast the QoS of web services. Geebelen et al. [12] come closest to a guarantee: a Petri-net simulation merges the monitored response times of the elementary services into a time series of the composition, and a kernel-based quantile estimator with an online offset correction predicts the quantile named in the SLO, such that the violation frequency converges to the agreed level. None of these works, however, learns QoS as a function of resource allocation, service configuration, and arrival rate. They can assess compositions of services as currently deployed, but not a composition that involves a configuration without observations; likewise, the guarantee of Geebelen et al. holds asymptotically in the number of observations (Challenge #2).

Fluxion [17] is the closest learned counterpart to our work. It trains one model per service (e.g., a deep neural network or a GP) that maps the service's configuration knobs, its observed states, and the latency percentiles of its downstream services to a latency percentile of the service itself. These models are chained into an inference graph that mirrors the service dependencies; after a service is scaled or replaced, only the model of that service is retrained. However, the inference graph propagates point estimates: Fluxion is evaluated by its mean absolute error, and its authors note that prediction errors accumulate along the graph in a way that is not trivial to derive analytically. A predicted P_{90} can therefore err in either direction, and the output does not reveal any uncertainty about whether a service model was queried at a configuration it has never seen (Challenge #2).

D. Summary

In summary, analytical models compose rigorously but need known parameters (i.e., access to the source code); top-down learned models support opaque microservices by handling them as black boxes, but they are only valid for the service topologies measured; and existing compositional models propagate predictions without any uncertainty. *Pegasus* (our approach) combines the strengths of these groups: it learns the execution time distribution of each black-box service from local observations, including the epistemic uncertainty

for unseen configurations, and composes per-service models into a probabilistic bound on the end-to-end latency.

III. Problem Definition

TODO: Also 50% autocompleted; I would shorten the descriptions and simplify the structure.

A. System Model

We model an application as a service composition S structured as a Directed Acyclic Graph (DAG) over a set of microservices $V = \{s_1, s_2, \ldots, s_K\}$, where directed edges $E \subseteq V \times V$ represent the paths along which items (e.g., requests or video frames) are passed on. For a sequential pipeline, this simplifies to $S = \{s_1 \to s_2 \to \cdots \to s_K\}$ with $K = |S|$ services.

Services. Each service $s_i \in V$ has a configuration $x_i = (r_i, c_i)$, where

- $r_i \in \mathcal{R}_i$ are its resource allocations (e.g., CPU quota, memory limit), and
- $c_i \in \mathcal{C}_i$ are its software configuration knobs (e.g., batch size, model resolution, number of workers).

A service processes one item at a time in the order of arrival; items that arrive while the service is busy wait in its queue. The *execution time* Y_i of s_i is the time (in seconds) from the start to the end of processing one item. Due to runtime variability (e.g., thread scheduling, input size), Y_i is a random variable whose unknown distribution $P_{Y_i|x_i}$ depends on the configuration of the service, but not on the composition in which the service runs. The *latency* of an item at s_i is its waiting time in the queue plus its execution time, $W_i = Q_i + Y_i$.

For each service, empirical evidence is available only as a sparse dataset of execution times observed at N_i configurations, $\mathcal{D}_i = \{(x_i^{(j)}, y_i^{(j)})\}_{j=1}^{N_i}$, collected while the service ran in isolation or within other compositions. Large regions of the configuration space $\mathcal{R}_i \times \mathcal{C}_i$ remain unobserved.

Workload. Items enter the composition at its first service according to an *external arrival process*, which we describe by the gap T (in seconds) between two consecutive arrivals: its mean defines the arrival rate $\lambda = 1/\mathbb{E}[T]$ (items per second), and its squared coefficient of variation $C_a^2 = \mathrm{Var}(T)/\mathbb{E}[T]^2$ the variability of the arrivals. The workload is a property of the application and not of an individual service; an item that leaves a service immediately enters the next one, so every later service receives the departures of its predecessor.

B. Problem Formulation

Let D_S denote the end-to-end latency of an item in the composition S under the configuration vector $\mathbf{x} = (x_1, \ldots, x_K)$, i.e., the time from its external arrival until it leaves the last service. Given a maximum permissible violation probability $\epsilon \in (0, 1)$ specified in an SLO, a latency bound d_S must satisfy

$$\mathbb{P}\left(D_S > d_S\right) \leq \epsilon, \tag{1}$$

which means that d_S is an upper bound on the $(1-\epsilon)$ quantile of the end-to-end latency.

Because the execution-time distributions are only known through models learned from $\mathcal{D}_1, \ldots, \mathcal{D}_K$, Eq. (1) cannot be asserted with certainty. A *Probabilistic Performance Guarantee* (PPG) therefore has two parameters: the violation probability ϵ, which covers the variability between items, and a confidence level $c \in (0, 1)$, which covers the uncertainty of the learned models. A PPG is a latency bound $d_S(\mathbf{x}, \epsilon, c)$ for which Eq. (1) holds with confidence c.

Problem Statement. Given a composition S, a configuration vector $\mathbf{x}$, an external arrival process, a violation probability ϵ, and a confidence level c, derive the PPG $d_S(\mathbf{x}, \epsilon, c)$ in a *zero-shot manner*, i.e., purely from the per-service datasets $\{\mathcal{D}_1, \ldots, \mathcal{D}_K\}$ and without any end-to-end measurement of S. The bound should be as tight as possible, since every unit of slack translates into over-provisioned resources.

C. Foundational Assumptions

To compose performance bounds across black-box services without end-to-end measurements, our framework operates under the following assumptions:

1) *Performance Isolation:* Services run in isolated execution environments (e.g., containers with strict CPU quotas and memory limits), so the execution time of a service is not affected by co-located services.
2) *Single-Server FIFO Services:* Each service processes one item at a time in the order of arrival; unprocessed items accumulate in an unbounded queue, and finished items are released immediately. Internal parallelism (e.g., the number of workers) is a configuration knob that changes the execution time, but not this queueing discipline.
3) *Load-Independent Execution Times:* The distribution of the execution time Y_i depends only on the configuration x_i; it depends neither on the arrival process nor on the services before s_i, and the execution times of different items at s_i are independent. This is what allows execution times observed in isolation, or in a different composition, to be reused for an unseen composition.
4) *Sub-Gaussian Execution Times:* The MGF of the execution time is bounded by that of a Gaussian,
$$M_{Y_i}(\theta) = \mathbb{E}\left[e^{\theta Y_i}\right] \leq \exp\left(\theta\, m_i + \frac{\theta^2}{2}\,\sigma_i^2\right), \quad \theta > 0, \tag{2}$$
with a mean m_i and a scale σ_i that depend on x_i. Gaussian execution times fulfill this with equality, and every bounded execution time fulfills it for some σ_i; in particular, a service with a timeout is sub-Gaussian. Execution times with heavier tails are discussed in Section VII-A.

   TODO: σ_i is then a variance *proxy*, which can exceed the standard deviation for skewed distributions. The spread we learn is not a proxy by construction; say how we obtain one (margin on the spread?).
5) *Renewal Arrivals:* The gaps T between external arrivals are independent, identically distributed, and independent of the execution times, and their Laplace transform

$\mathbb{E}[e^{-\theta T}]$ is known. This covers, for instance, Poisson arrivals, periodic arrivals with jitter, and gamma-distributed gaps of any C_a^2.

6) *Arrivals Inside the Composition:* The framework does not model the departure process of a service explicitly. A service that is not the first of the composition is bounded as if it received the external arrival process, refined by the minimum spacing that its predecessor enforces between two departures (Section IV-B). This is conservative when the external arrivals are at least as variable as the execution times, as with Poisson arrivals; for nearly periodic arrivals combined with highly variable execution times and high utilization, the departures of a service can be burstier than the external arrivals.

TODO: Decide after the prototype: keep this assumption, or replace it with a proven bound on the departures (looser).

Independence between the execution times of an item at *different* services is not assumed in general; it is only required by one of the two composition variants (Section IV-C).

IV. Pegasus Framework

TODO: Way too long. Shorten especially the intro, but also the subsections, and try to remove some equations.

Given a pipeline S, our methodology derives a Probabilistic Performance Guarantee (PPG) for S. Such a PPG characterizes the performance of a service or service composition together with its inherent runtime variability and the uncertainty resulting from limited empirical evidence. Conceptually, a PPG can be established for individual services (i.e., a single-service pipeline with $|S| = 1$); these local bounds are then composed into end-to-end performance guarantees for multi-service pipelines. Notably, a PPG does not prescribe any optimization objective or algorithm. Instead, it provides a fundamental performance guarantee that enables downstream frameworks to optimize for objectives such as latency, throughput, or cost.

Our methodology consists of three main steps, as depicted in Figure 2: (1) learning a probabilistic model of the execution time of each individual service from empirical observations, (2) deriving a latency bound for each service from its model and the workload, and (3) composing these bounds into a PPG for the complete service pipeline. The first step is the only one that requires data, and it only requires data of the individual services. Everything that depends on the composition, i.e., which services follow each other and how many items arrive, enters analytically in the second and third step. This separation is what makes the guarantee available for compositions that were never executed.

A. Learning

The first step trains, for each service s_i, a probabilistic model that predicts the execution time Y_i of an item from the configuration x_i of the service. Two constraints apply to this step: (1) the scaling behavior of services is rarely

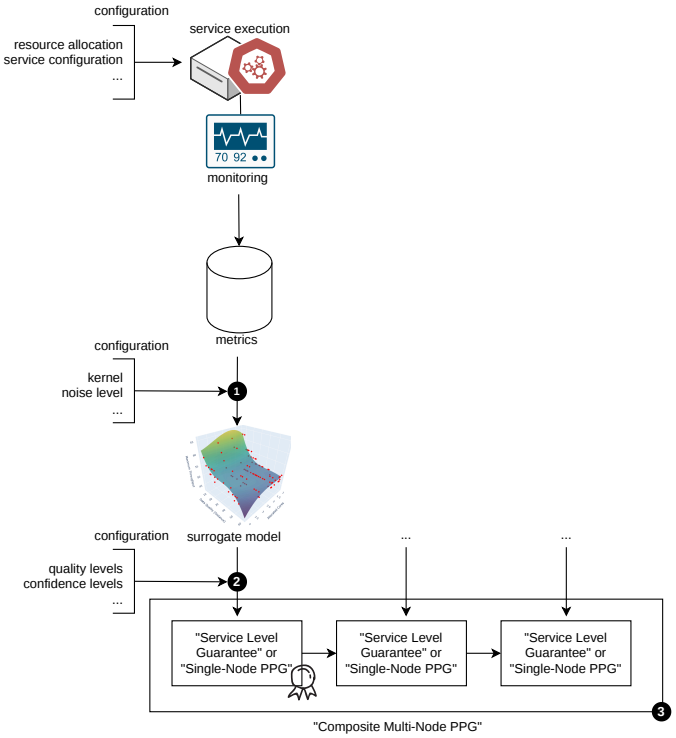


Fig. 2: High-level overview of our three-step methodology for creating end-to-end performance guarantees of arbitrary service composition and configurations: ❶ we model service behavior through a surrogate function, ❷ we extract probabilistic performance guarantees (PPGs) around individual services, and ❸ we calculate the expected performance levels across service compositions by concatenating their PPGs.

linear [3], e.g., performance saturates after providing sufficient RAM, while increasing parallelism can actually degrade performance due to scheduling overhead; and (2) services live in containers, whose source code can rarely be analyzed to make assumptions about their functional behavior. Gaussian Processes (GPs) satisfy both constraints, as they are non-parametric and do not impose a structure on the learned function.

We train one GP per service on its dataset $\mathcal{D}_i$, with a Matérn-5/2 kernel and an additive noise term. For a configuration x_i, which may be unobserved, the GP provides three quantities:

- the predicted mean execution time $\mu_i(x_i)$;
- the *epistemic* standard deviation $\sigma_{\mathrm{ep},i}(x_i)$, i.e., the uncertainty of the model about the mean execution time at x_i, which is large far away from the observed configurations and shrinks with more data; and
- the *aleatoric* standard deviation $\sigma_{\mathrm{al},i}(x_i)$, i.e., the scatter of the execution times of individual items around their mean, which is a property of the service and does not shrink with more data.

The following two steps treat these two kinds of uncertainty differently. In the remainder of this section, we omit the argument x_i where it is clear from the context.

The number of observations per service is small in our setting, so we use exact GP inference; its cost is cubic in the number of observations N_i. For services that accumulate large amounts of operational data, the exact GP can be replaced by a sparse approximation (e.g., inducing-point methods) without changing the following steps, which only rely on the three quantities above.

B. Service Bounds

The second step turns the model of a service into a bound on its latency $W_i = Q_i + Y_i$ under a given workload: a value $d_i(\epsilon)$ that the latency of an item exceeds with probability at most ϵ. Unlike the execution time, the latency includes the waiting time Q_i in the queue of the service, and therefore depends on how many items arrive.

Pessimistic Execution-Time Model. The two kinds of uncertainty act differently on a queue. The aleatoric scatter is independent from item to item, so over a busy period of many items it partly averages out. The epistemic error does not: if the model underestimates the mean execution time by 5 ms, *every* item at that configuration takes 5 ms longer, and the work of a busy period of 100 items grows by 500 ms. Adding both variances into the variance of a single item would treat this shared error as if it averaged out, and would underestimate the queue. We therefore let the epistemic uncertainty shift the mean, and keep only the aleatoric uncertainty as the variance per item:

$$\mu_i^+ = \mu_i + z_c\,\sigma_{\mathrm{ep},i}, \qquad Y_i \sim \mathcal{N}\big(\mu_i^+,\, \sigma^2_{\mathrm{al},i}\big)\,, \tag{3}$$

where z_c is the c-quantile of the standard normal distribution (e.g., $z_c \approx 1.64$ for $c = 0.95$). According to the model, the true mean execution time lies below the *pessimistic mean* μ_i^+ with probability c; this is where the confidence level of the PPG enters. With Eq. (2), the MGF of the execution time is $M_i(\theta) = \exp(\theta\mu_i^+ + \frac{\theta^2}{2}\sigma^2_{\mathrm{al},i})$.

Waiting Time. The waiting time of an item at a FIFO service is the largest backlog of work that the items before it left behind. For a service that receives arrivals with independent gaps T, Kingman's martingale bound [16] states that this backlog has an exponentially decaying tail,

$$\mathbb{P}\left(Q_i > w\right) \le e^{-\theta_i^* w}, \qquad \theta_i^* = \sup\left\{\theta > 0 : M_i(\theta)\,\mathbb{E}\big[e^{-\theta T}\big] \le 1\right\}. \tag{4}$$

The *decay rate* θ_i^* (in $1/\mathrm{s}$) combines everything that makes a queue long: it decreases with a larger mean execution time, with a larger variance of the execution time, with a higher arrival rate, and with more variable arrivals. For Poisson arrivals at rate λ, for instance, $\mathbb{E}[e^{-\theta T}] = \lambda/(\lambda+\theta)$. If no $\theta > 0$ satisfies the condition, the service counts as overloaded and has no finite bound. Since the condition is evaluated with the pessimistic mean μ_i^+, this can also happen for a service that is stable but whose mean execution time is not known precisely enough.

Latency. The waiting time of an item depends only on the items before it, and is therefore independent of its own execution time. Combining Eq. (4) with the Gaussian execution time of Eq. (3) gives the tail of the latency in closed form,

$$\begin{aligned}\mathbb{P}\left(W_i > d\right) \le\; & \bar{\Phi}\left(\frac{d-\mu_i^+}{\sigma_{\mathrm{al},i}}\right) \\ & + e^{-\theta_i^*(d-\mu_i^+)+\frac{1}{2}(\theta_i^*\sigma_{\mathrm{al},i})^2}\;\Phi\left(\frac{d-\mu_i^+-\theta_i^*\sigma^2_{\mathrm{al},i}}{\sigma_{\mathrm{al},i}}\right),\end{aligned} \tag{5}$$

where Φ is the standard normal distribution function and $\bar{\Phi} = 1 - \Phi$. The first term is the probability that the execution time alone exceeds d; the second term accounts for the waiting time. The *service bound* $d_i(\epsilon)$ is the smallest d for which the right-hand side is at most ϵ; the right-hand side decreases in d, so $d_i(\epsilon)$ is found by bisection. Eq. (4) and the composition of Section IV-C only use the bound of Eq. (2); Eq. (5) additionally uses the Gaussian shape for the item's own execution time.

TODO: For sub-Gaussian but non-Gaussian Y_i: split $d = d_Y + d_Q$ and bound $\mathbb{P}(Y_i > d_Y) \le e^{-(d_Y-\mu_i^+)^2/(2\sigma_i^2)}$ and $\mathbb{P}(Q_i > d_Q) \le e^{-\theta_i^* d_Q}$ separately. Slightly looser; footnote or appendix.

Services Inside an Application. Only a limited set of fronted services receives an external arrival process; every subsequent service s_i receives the departures of its predecessor s_{i-1}. We bound such a service with the external arrival process in Eq. (4) (Assumption 6 in Section III-C), and tighten the bound with a property that holds for every pipeline: s_{i-1} processes one item at a time, so two consecutive departures of s_{i-1} cannot be closer together than one execution time of s_{i-1}. If s_{i-1} is slower than s_i, the service s_i can therefore hardly build up a queue, whatever the external arrival rate is. Applying Kingman's bound to the execution times of s_i against this *minimum spacing* gives a second decay rate,

$$\theta_i^{\mathrm{s}} = \frac{2\left(\mu^-_{i-1} - \mu_i^+\right)}{\sigma^2_{\mathrm{al},i} + \sigma^2_{\mathrm{al},i-1}}, \qquad \mu^-_{i-1} = \mu_{i-1} - z_c\,\sigma_{\mathrm{ep},i-1}, \tag{6}$$

which is positive only if s_{i-1} is slower than s_i even when s_{i-1} enters with its *optimistic* mean μ^-_{i-1} (a faster predecessor spaces the items less) and s_i with its pessimistic mean. Both tails bound the same waiting time, so the service bound uses the faster decay, i.e., $\max(\theta_i^*, \theta_i^{\mathrm{s}})$ in place of θ_i^* in Eq. (5). Eq. (6) treats the execution times of consecutive items at the two services as independent.

C. Composition

The third step composes the models of the individual services into the PPG of the pipeline S. We provide two variants, which trade assumptions against tightness and which we both evaluate in Section VI.

Composition per Service (*Pegasus Node*). The end-to-end latency is the sum of the latencies at the K services, $D_S = \sum_{i=1}^{K} W_i$. An item can only exceed the sum of the service bounds if it exceeds at least one of them. Hence, if every service is bounded at the violation probability ϵ/K,

$$d_S = \sum_{i=1}^{K} d_i\Big(\frac{\epsilon}{K}\Big) \quad \Rightarrow \quad \mathbb{P}\left(D_S > d_S\right) \le \sum_{i=1}^{K} \mathbb{P}\left(W_i > d_i\Big(\frac{\epsilon}{K}\Big)\right) \le \epsilon. \tag{7}$$

This union bound requires nothing about how the latencies at different services depend on each other. It therefore remains

valid if an item that is slow at one service is also slow at the next one (e.g., a large video frame), and although the queues of consecutive services are coupled. Its price is that every service is bounded further in the tail as the pipeline grows, and that every service pays for its own queue as if it stood alone, apart from the minimum spacing of Eq. (6). Each service bound is one closed formula, so the cost of evaluating d_S grows linearly with K.

Composition per Pipeline (*Pegasus SNC*). Stochastic Network Calculus (SNC) [8] composes the *service* that the nodes offer instead of their bounds: the pipeline is treated as one system that receives the external arrivals once, and what happens between the services is absorbed into the composition. In the MGF domain, the end-to-end latency of a pipeline of FIFO services obeys

$$\mathbb{P}\left(D_S > d\right) \leq e^{-\theta d} \prod_{i=1}^{K} \frac{M_i(\theta)}{1-\rho_i(\theta)}, \qquad \rho_i(\theta) = M_i(\theta)\,\mathbb{E}\left[e^{-\theta T}\right] < 1, \tag{8}$$

for every $\theta > 0$, where each factor is the latency moment of one service: its execution-time MGF, inflated by the queue that the load $\rho_i(\theta)$ causes. Solving for the latency at the violation probability ϵ and choosing the best θ yields

$$d_S = \min_{\theta>0} \frac{1}{\theta}\left[\sum_{i=1}^{K} \ln \frac{M_i(\theta)}{1-\rho_i(\theta)} - \ln \epsilon\right]. \tag{9}$$

In contrast to Eq. (7), the violation probability is not split: the term $-\ln\epsilon$ is paid once for the whole pipeline, which is the "pay bursts only once" property of network calculus. Moreover, Eq. (9) makes no assumption about the arrivals inside the pipeline, so Assumption 6 is not needed. In return, a single parameter θ is shared by all services, so one service close to saturation loosens the bound of the whole pipeline, and the product in Eq. (8) requires that the execution times of an item are independent between the services. If they are not, Hölder's inequality restores the validity for any dependence: service s_i enters Eq. (8) with its variance multiplied by an exponent $p_i \geq 1$, where $\sum_i 1/p_i = 1$. This makes the bound looser and its evaluation more expensive, because the exponents are tuned per configuration; we report it as an ablation.

TODO: What needs to be shared in the two variants?

Resulting Guarantee.

Both variants return a latency bound that satisfies Eq. (1) whenever the pessimistic mean of every service lies above its true mean execution time. Each service fulfills this with confidence c, according to its model. For the whole pipeline, all K services must fulfill it simultaneously, which holds with confidence of at least $1 - K(1-c)$; choosing the per-service level as $1-(1-c)/K$ would restore the confidence c for the pipeline, at the price of larger pessimistic means. We apply c per service and report in Section VI how often the resulting bounds cover the measured latency quantile of the pipeline.

TODO: Decide: keep c per service (current implementation) or split it over the services.

Why This Combination. The three steps depend on each other. Point-estimate models (e.g., neural networks or gradient-boosted trees) provide no epistemic uncertainty and thus no pessimistic mean, which is what protects the bound at configurations that were never observed. The GP provides it, and separates it from the scatter between items. Its Gaussian form gives the MGF of Eq. (2) in closed form, and the MGF is exactly the quantity from which both the martingale bound and SNC derive the tail of a queue. As a result, the service bounds and their composition are evaluated from closed-form expressions, without simulating the pipeline and without sampling.

V. Implementation

TODO: Needs some introduction and cutting.

Service Models. Each service model is an exact GP with a Matérn-5/2 kernel and an additive white-noise term, fitted on standardized configurations. The epistemic variance of a prediction is the variance of the latent function, and the aleatoric variance is the fitted noise level.

Training on Run Statistics. A benchmark run of a real service processes many items at one configuration. Instead of a single item per run, we use two statistics of each run as training targets: the mean of its execution times, and their spread, defined as (95th percentile − median) / 1.645, i.e., the standard deviation of a Gaussian with that median and 95th percentile. The mean of a run is a far less noisy observation of the mean execution time than a single item; in turn, it no longer reveals how much single items scatter, so a second GP per service predicts the spread and provides $\sigma_{\mathrm{al},i}$. Both GPs are fitted on the logarithm of their target, because the knobs of a real service act multiplicatively on its execution time (e.g., a larger model at a higher resolution), which becomes additive on the logarithm. The shift of Eq. (3) is then relative, $\mu_i^+ = \mu_i\, e^{z_c \sigma_{\mathrm{ep},i}}$ with $\sigma_{\mathrm{ep},i}$ on the logarithmic scale, where $\sigma_{\mathrm{ep},i}$ also contains the scatter between repeated runs of the same configuration. The predicted spread enters the bounds without a margin of its own.

Evaluating the Bounds. The decay rate θ_i^* of Eq. (4) and the minimum in Eq. (9) are evaluated on a fixed logarithmic grid of θ; any θ below θ_i^* keeps Eq. (4) valid, so the grid can only make a bound larger. The service bound $d_i(\epsilon)$ is found by bisection on Eq. (5) and returns the upper end of the final interval.

Discretization of the Configuration Space. Numerous service knobs could actually be scaled on a fine-grained Integer scale, for example, the scheduled CPU quota is configured in *microseconds*, or the resolution of video frames in *pixel*. Nevertheless, Cloud systems commonly offer only a small set of discretized configurations, around which the performance predictions are more accurate [3]. We hypothesize that the granularity of the discretization might improve the convergence of accurate PPGs, while trading off solution optimality. Not sure if we'd analyze this now.

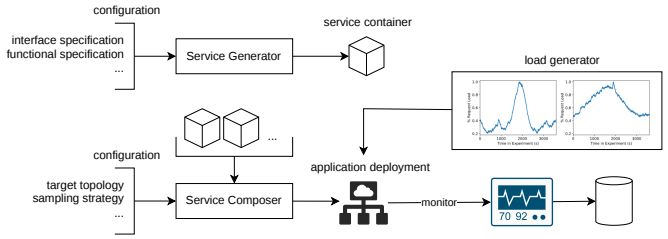


Fig. 3: High-level architecture of the experimental environment: the *service generator* creates containerized services according to a desired interface and functionality; the *service composer* ...

VI. Evaluation

A. Experimental Setup

Service Generation. To evaluate generalization across structural dimensions, we evaluate unseen service configurations, unseen service compositions, and their combination. This requires an ample set of services and service compositions; for this, we combine an existing set of microservices and configuration interfaces from prior work [24] and augment it through LLM-based code generation.

Sampling Strategies. To evaluate the impact of training data collection strategies on bound calibration, empirical observations will be gathered under three *sampling strategies*: (1) uniform random sampling across the configuration space, (2) active learning under different acquisition functions (e.g., uncertainty minimization of value maximization), and (3) exhaustive grid search, which we must prune at some point.

Baselines. We compare *Pegasus* against five baselines that all operate under the same constraint: each method is trained only on observations of the individual services and never on a run of the evaluated composition. Hence, methods produce an end-to-end latency for an *unseen* composition S with $K = |S|$ services, which we compare with the ground truth, i.e., the measured $(1-\epsilon)$ quantile of the end-to-end latency of S. The baselines form three groups, which differ in what they observe per service and in how they account for queueing:

- **Execution-time models without queueing (*GP Sum*, *DNN Sum*):** one model per service predicts the mean and the standard deviation of the execution time from the service configuration; the end-to-end latency is the $(1-\epsilon)$ quantile of the sum of these execution times, assuming that they are Gaussian and independent between services. *GP Sum* uses the same GPs as *Pegasus*, so the two differ only in the composition; *DNN Sum* replaces each GP with a feed-forward neural network that reports the standard deviation of its training residuals. Both ignore the arrival process and thus represent the naive composition of per-service predictions.

  TODO: As in Fluxion [17] we must argue that this is a common method for predicting end-to-end latency.

- **Queueing formula (*Kingman*):** the predicted mean and variance of the GPs enter Kingman's approximation of the mean waiting time in a G/G/1 queue [16], combined with an exponential waiting-time tail [14]. This gives the $(1-\epsilon)$ latency quantile of each service, and the quantiles are added. *Kingman* represents classical queueing analysis on top of a learned model: it accounts for queueing, but it is an approximation and not a bound, and it takes the GP predictions at face value, without a margin for epistemic uncertainty.
- **Learned latency quantiles (*Quantile Union*, *Conformal*):** instead of execution times, these baselines learn the latency of a service under load. Each service is monitored in isolation under the arrival process of the workload, and one GP per service predicts its $(1-\epsilon/K)$ latency quantile from the service configuration; the K predictions are added, which splits the violation budget ϵ over the services as a union bound. This corresponds to the modular learning of latency percentiles in *Fluxion* [17], with every model restricted to data of its own service. *Quantile Union* uses these point predictions directly. *Conformal* adds a split-conformal step: a part of the training configurations is set aside, and each prediction is multiplied by a correction factor calibrated on that part, such that it covers the latency quantile of the service with a chosen confidence. Both need a load test per service and training configuration, whereas *Pegasus* needs execution times only, and both assume that a service receives the same arrival process inside the composition as in isolation.

As a reference, we report an *Oracle*, the union bound over the *measured* $(1-\epsilon/K)$ latency quantiles of the services inside the composition. It uses the ground truth and is therefore not a competitor; it shows the tightest result that any method splitting the budget ϵ over the services can reach, so the remaining distance to a method is caused by its per-service bounds and not by the union bound.

Besides the baselines, we ablate the composition step of *Pegasus* on identical GP models: (1) the union bound over per-service bounds, with Chernoff-based or martingale-based bounds, each with and without propagating the departure process of the preceding service, and (2) the network service curve, with and without Hölder's inequality, which extends its validity to dependent services. Finally, the experiment on fine-tuning with end-to-end samples (Section VI-C1) is the only one that includes baselines trained on runs of the composition itself: the sum of per-service latency quantiles measured inside the composition, and a *Fluxion*-style chain of GPs that passes predicted latency percentiles from one service to the next [17].

B. Coverage and Tightness of Service Bounds

1) Base Case: This will use most of the baselines from above to evaluate the maturity of our ideas vs. existing techniques.

2) Varying Evidence per Service: This makes use of the presented sampling strategies. The services are executed somewhere under an arbitrary service composition that is different from the target composition. We gradually train the PPGs with more of this data and see if this has an impact on the quality of the unseen target composition.

3) Varying Chain Depth: We evaluate how the validity, empirical coverage rate, and tightness of the derived PPGs scale with pipeline depth. By evaluating synthetic and real chains across varying lengths $|S| \in \{1, 2, 5, 10, 20, 50, 100\}$, we assess whether the composed bounds remain sufficiently tight and actionable for downstream optimization without suffering from severe bound degradation.

PROBLEM: We promise DAGs, but in our current state of evaluations we only have chains. This is not enough and we must have some more complex DAGs.

TODO: Before we get Milos' results on the hierarchical application, I can look into synthetic data.

4) Varying Load and Arrival Pattern: Here we will use different arrival pattern (e.g., periodic, random (= Poisson), and batching) and different arrival rates (e.g., diurnal or bursty with custom range). I already explored this once and found that we can characterize each of these arrival processes through the squared coefficient of variation (C_a^2) as defined by

$$C_a^2 = \frac{\mathrm{Var}(X)}{(\mathbb{E}[X])^2} = \frac{\sigma^2}{\mu^2} \tag{10}$$

5) Varying Tail Behaviors: Here I can look at services that have a different tail behavior (e.g., increasingly heavy/light tails) and how this impacts the quality of the guarantee.

TODO: Add a timeout variant: the same heavy-tailed services with execution times cut off at a timeout. Expectation: the bound fails for the heaviest tails without a timeout and holds with one. The simulator has the lognormal tail, but no cutoff yet.

6) Varying Percentiles: Optionally, I could also look into different percentile guarantees and show where each method is more/less accurate.

C. Iterative Optimization of Service Bounds

I would integrate this into a use case, where we have to compose increasingly better applications using this method.

1) Fine-Tuning Bounds with E2E Samples: Our focus is giving a rough guarantee *before* deploying it e2e, so this means the best we can do as soon as it goes e2e is to show that our model provides an initial accuracy boost that likely vanishes over time.

2) Inferring Faster Service Compositions: This can go into two directions, first, we can optimize the service configuration according to the model, and second, we can improve the service composition.

For the first one, we can take *Fluxion* [17] (cf. Section 5.1.3) as inspiration, where they also used the model paired with an external optimizer to show how their model enables better performance optimization. They simply compared the different guarantee models (theirs, a plain GP, and a DNN) for coupled with different optimizers; this we can do as well, and it is also a nice cut between the guarantee (our contribution) and the external optimizer.

For the second one, it might be a bit more complex because we have not talked a lot about the composition mechanism, but I would design it like the first one: we assume some composition mechanism that is external to us and we gradually show that by using our guarantees it can create better service composition. I think that this is the real deal because it repeatedly looks into unseen configurations, but it also the most complex evaluation. In any case, I think that a genetic algorithm could be the best choice here, because it builds a population of potential solutions and then iteratively improves them through sampling. But we do NOT have to include multiple rounds of benchmarking, but only one. The distinction would be:

- **single round**: we build a population of compositions and grade them according to different guarantees (ours vs. baselines). We repeat this step in a genetic algorithm until we have a solution for every guarantee. Then we benchmark these solutions and look at their fitness.
- **multiple round**: as above, but we use the empirical samples we captured from the benchmark and run the genetic algorithm (planning) and benchmarking again and again until we converge to a satisfying result. This we keep for future work at NSDI hehe

VII. Discussion

@Víctor most likely

A. Limitations

Heavy-Tailed Execution Times. The performance of distributed systems is known to show heavy-tailed behavior, e.g., during garbage collection. *Pegasus* requires sub-Gaussian execution times (Assumption 4). If a Gaussian is fitted to a service with a heavier tail, the bound stays accurate where queueing dominates the latency, because a long queue depends mainly on the mean and the variance of the execution time; it becomes optimistic where the tail of a single execution dominates, i.e., at low utilization, for very regular arrivals, and for high percentiles. The per-service composition is affected more strongly in deep pipelines, as it bounds every service at ϵ/K.

Two extensions keep the framework applicable. First, services in production enforce timeouts; an execution time that is cut off at a timeout τ is bounded and therefore sub-Gaussian, so the guarantee holds for the service as deployed.

TODO: Draft of the mechanism: with a hard timeout, $Y_i \leq \tau$ always, and Bennett's inequality bounds the MGF from the variance and the range, which is tighter than a Gaussian with the range as scale. Without a hard timeout: cap at a high quantile and reserve a part of ϵ for items beyond the cap; this needs a proof for the items in the same busy period.

Second, network calculus has been extended to heavy-tailed processes with power-law instead of exponential envelopes [18]; the resulting bounds decay polynomially, are considerably looser, and lose the closed forms that make *Pegasus* fast to evaluate, which is why we do not adopt them.

Sampling from Evaluation Rounds. The idea was that we can drastically simplify experiments by not propagating decisions (e.g., *where* to explore) back to the services, but

instead use the grading function to sample from all conducted runs. Given the coverage of the solution space, we can bound the inaccuracy this creates by finding the largest distance between two points, or a point and the parameter bounds.

VIII. Conclusion & Future Work

@Víctor most likely

References

[1] Magrans de Abril, I., Sedlak, B., Pujol, V.C., Dustdar, S.: Composable Autonomic Offerings for Decentralized Service Provisioning in the Computing Continuum. In: Proceedings of the Workshop on Advanced Tools, Programming Languages, and PLatforms for Implementing and Evaluating algorithms for Distributed systems. pp. 1–9. ApPLIED'26 (Jul 2026). doi:10.1145/3820355.3820395

[2] Aiello, M.: A Paradigm Shift in Service Research: The Case of Service Composition. IEEE Transactions on Services Computing **18**(3), 1213–1215 (May 2025). doi:10.1109/TSC.2025.3552345

[3] Alipourfard, O., Liu, H.H., Chen, J., Venkataraman, S., Yu, M., Zhang, M.: CherryPick: Adaptively Unearthing the Best Cloud Configurations for Big Data Analytics. In: 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17). pp. 469–482 (2017)

[4] Duan, Q.: Modeling and performance analysis for composite network–compute service provisioning in software-defined cloud environments. Digital Communications and Networks **1**(3), 181–190 (Aug 2015). doi:10.1016/j.dcan.2015.05.003

[5] Esashi, A., Lertpongrujikorn, P., Kato, S., Salehi, M.A.: Action Engine: Automatic Workflow Generation in FaaS. Future Generation Computer Systems **174**, 107947 (Jan 2026). doi:10.1016/j.future.2025.107947

[6] Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., Zhang, J.M.: Large Language Models for Software Engineering: Survey and Open Problems. In: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE). pp. 31–53 (May 2023). doi:10.1109/ICSE-FoSE59343.2023.00008

[7] Fanjiang, Y.Y., Syu, Y., Kuo, J.Y.: Search based approach to forecasting QoS attributes of web services using genetic programming. Information and Software Technology **80**, 158–174 (Dec 2016). doi:10.1016/j.infsof.2016.08.009

[8] Fidler, M., Rizk, A.: A Guide to the Stochastic Network Calculus. IEEE Communications Surveys & Tutorials **17**(1), 92–105 (2015). doi:10.1109/COMST.2014.2337060

[9] Franks, G., Al-Omari, T., Woodside, M., Das, O., Derisavi, S.: Enhanced Modeling and Solution of Layered Queueing Networks. IEEE Transactions on Software Engineering **35**(2), 148–161 (Mar 2009). doi:10.1109/TSE.2008.74

[10] Gallotti, S., Ghezzi, C., Mirandola, R., Tamburrelli, G.: Quality Prediction of Service Compositions through Probabilistic Model Checking. In: Becker, S., Plasil, F., Reussner, R. (eds.) Quality of Software Architectures. Models and Architectures. pp. 119–134. Springer, Berlin, Heidelberg (2008). doi:10.1007/978-3-540-87879-7_8

[11] Gandhi, A., Harchol-Balter, M., Das, R., Lefurgy, C.: Optimal power allocation in server farms. In: Proceedings of the eleventh international joint conference on Measurement and modeling of computer systems. pp. 157–168. SIGMETRICS '09, Association for Computing Machinery, New York, NY, USA (Jun 2009). doi:10.1145/1555349.1555368

[12] Geebelen, D., Geebelen, K., Truyen, E., Michiels, S., Suykens, J.A.K., Vandewalle, J., Joosen, W.: QoS prediction for web service compositions using kernel-based quantile estimation with online adaptation of the constant offset. Information Sciences **268**, 397–424 (Jun 2014). doi:10.1016/j.ins.2013.12.063

[13] Guo, Y., Wu, M., Cao, Y., Wei, J., Ran, D., Yang, W., Xie, T.: BackendForge: Benchmarking Agentic End-to-End Code Generation with Backend Services (Jul 2026). doi:10.48550/arXiv.2607.11042, arXiv:2607.11042 [cs.SE]

[14] Harchol-Balter, M.: Performance modeling and design of computer systems: queueing theory in action. Cambridge University Press (2013)

[15] Iyer, R., Pedrosa, L., Zaostrovnykh, A., Pirelli, S., Argyraki, K., Candea, G.: Performance Contracts for Software Network Functions. In: 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19). pp. 517–530. USENIX Association (Feb 2019)

[16] Kingman, J.F.C.: The single server queue in heavy traffic. Mathematical Proceedings of the Cambridge Philosophical Society **57**(4), 902–904 (Oct 1961). doi:10.1017/S0305004100036094

[17] Liang, C.J.M., Fang, Z., Xie, Y., Yang, F., Li, Z.L., Zhang, L.L., Yang, M., Zhou, L.: On Modular Learning of Distributed Systems for Predicting End-to-End Latency. pp. 1081–1095 (2023)

[18] Liebeherr, J., View Profile, Burchard, A., View Profile, Ciucu, F., View Profile: Delay Bounds in Communication Networks With Heavy-Tailed and Self-Similar Traffic. IEEE Transactions on Information Theory **58**(2), 1010–1024 (Feb 2012). doi:10.1109/TIT.2011.2173713

[19] Marinescu, D.C.: Chapter 10 - Complex Systems and Self-Organization. In: Cloud Computing, pp. 301–315. Morgan Kaufmann, Boston (Jan 2013). doi:10.1016/B978-0-12-404627-6.00010-5

[20] Meflah, W., Brabra, H., Acheli, M., Douma, H.B., Sellami, M., Gaaloul, W., Zeghlache, D.: From Natural Language to TOSCA: Leveraging LLMs for Automated Service Composition. In: Aiello, M., Deng, S., Murillo, J.M., Georgievski, I., Benatallah, B., Wang, Z. (eds.) Service-Oriented Computing. pp. 3–18. Springer Nature, Singapore (2026). doi:10.1007/978-981-95-5012-8_1

[21] Papazoglou, M.P., Georgakopoulos, D.: Introduction: Service-oriented computing. Communications of the ACM **46**(10), 24–28 (Oct 2003). doi:10.1145/944217.944233

[22] Rzadca, K., Findeisen, P., Swiderski, J., Zych, P., Broniek, P., Kusmierek, J., Nowak, P., Strack, B., Witusowski, P., Hand, S., Wilkes, J.: Autopilot: workload autoscaling at Google. In: Proceedings of the Fifteenth European Conference on Computer Systems. pp. 1–16. EuroSys '20, New York, NY, USA (Apr 2020). doi:10.1145/3342195.3387524

[23] Sedlak, B., Pujol, V.C., Motlagh, N.H., Morabito, R., Wilhelmi, F., de Abril, I.M., Murturi, I., Bellalta, B., Donta, P.K., Dustdar, S.: Large-Scale Sensor Systems for Sentient Environments: Emerging Research Directions. IEEE Journal of Selected Areas in Sensors pp. 1–15 (2026). doi:10.1109/JSAS.2026.3716613

[24] Sedlak, B., Raith, P., Morichetta, A., Pujol, V.C., Dustdar, S.: Multi-Dimensional Autoscaling of Stream Processing Services on Edge Devices. IEEE Transactions on Services Computing pp. 1–13 (2026). doi:10.1109/TSC.2026.3711142

[25] Venkataraman, S., Yang, Z., Franklin, M., Recht, B., Stoica, I.: Ernest: Efficient Performance Prediction for Large-Scale Advanced Analytics. In: 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16). pp. 363–378 (2016)

[26] Wilhelm, R., Engblom, J., Ermedahl, A., et al.: The worst-case execution-time problem—overview of methods and survey of tools. ACM Transactions on Embedded Computing Systems (TECS) **7**(3), 1–53 (2008)

[27] Wu, Q., Zhang, M., Zheng, R., Lou, Y., Wei, W.: A QoS-Satisfied Prediction Model for Cloud-Service Composition Based on a Hidden Markov Model. Mathematical Problems in Engineering **2013**(1), 387083 (2013). doi:10.1155/2013/387083

[28] Yu, G., Chen, P., Zheng, Z.: Microscaler: Cost-Effective Scaling for Microservice Applications in the Cloud With an Online Learning Approach. IEEE Transactions on Cloud Computing **10**(2), 1100–1116 (Apr 2022). doi:10.1109/TCC.2020.2985352

Potential Appendix

- List of the services and their states and configuration knobs in more detail. This can be clusters of service types, as for *Fluxion* in [17].
- Description of the code artifact, or at least a pointer to the extended description in the README there.